JAMIA MILLIA ISLAMIA

(A Central University, New Delhi)

Department of Computer Science & Engineering

B.Tech. (Computer Engineering) — Semester V — Session 2026–27

CSC-504: SOFTWARE ENGINEERING

ASSIGNMENT 1

Requirement Analysis & Specification · Software Design · Software Reliability

General Instructions

1. All questions are compulsory. Attempt every part.
2. Show complete, step-by-step working for all numerical problems; answers without working carry no credit.
3. All diagrams must be neatly drawn and clearly labelled (hand-drawn diagrams are acceptable).
4. Parts D and E are self-study topics. Short primers are provided so you can learn and solve them on your own — use the formulas and constants given in the primer.
5. State any assumptions clearly. Write answers in your own words.

PART A — Software Design & Reliability: Short-Answer Questions

Answer each question briefly, in two to three sentences.

A1. Define problem analysis. Why must it be carried out before software design begins?

A2. State any three differences between Function-Oriented Design (FOD) and Object-Oriented Design (OOD).

A3. Explain the design principles of modularity, abstraction, and information hiding, and state why each is important.

A4. What is a structure chart? Define fan-in and fan-out, and state which is desirable for a reusable module.

A5. What is software prototyping? Explain how it helps during requirements analysis.

A6. List any four characteristics of a good Software Requirements Specification (SRS) document.

A7. List any four golden rules/principles of good User-Interface (UI) design.

A8. Distinguish between a fault, an error, and a failure in software reliability, giving one example of each.

A9. Define MTTF, MTTR and MTBF, give the relation among them, and explain how reliability differs from availability.

A10. State the five maturity levels of the SEI Capability Maturity Model (CMM), in order.

A11. What is the ISO 9001 standard, and how is it related to software quality assurance?

PART B — Software Design: Diagrams

Draw the required diagrams using correct, standard notation and label every component.

B1. For an **Online Food Delivery System**, draw the **Level-0 (context) DFD** and a **Level-1 DFD**. Show at least four processes (customer registration, place order, process payment, track delivery) and all relevant data stores. Ensure the two levels are balanced.

B2. Draw a **Use Case Diagram** for an **Online Shopping (e-commerce) System**. Include the actors (Customer, Payment Gateway) and at least four use cases (browse products, add to cart, place order, make payment), and show one <<include>> relationship (e.g. “validate payment”).

B3. Draw a **Structure Chart** for a **Payroll Processing System**, showing the top-level module and its subordinate modules (read employee data, calculate gross salary, calculate deductions, calculate net salary, print pay-slip). Mark the data couples and any control flag passed between modules.

B4. Draw an **Entity-Relationship (ER) Diagram** for a **Hospital Management System**. Include at least four entities (Patient, Doctor, Appointment, Department), their key attributes, the relationships among them, and the cardinality (1:1, 1:N, M:N) of each relationship.

B5. Draw a **Class Diagram** (object-oriented design) for a **University Course-Registration System**. Include at least four classes (Student, Course, Faculty, Registration) with their attributes and methods, the associations among them, and the multiplicities.

PART C — Cohesion & Coupling

Attempt all questions. For the classification tables, write only the correct type against each item.

C1. Define cohesion and coupling. Why is a design with HIGH cohesion and LOW coupling considered good? Give two reasons.

C2. List the seven types of cohesion in order, from the weakest (worst) to the strongest (best).

C3. List the types of coupling in order, from the worst (highest) to the best (lowest).

C4. Differentiate between stamp coupling and data coupling, with one example of each.

C5. Differentiate between functional cohesion and logical cohesion, with one example of each.

C6. Identify the TYPE of cohesion in each module below (functional, sequential, communicational, procedural, temporal, logical, or coincidental):

#	**Module description**
(a)	A module whose only task is to compute the square root of a number.
(b)	A module that reads a record, validates it, then updates it — each step uses the output of the previous one.
(c)	From one employee record, a module computes both the income tax and the net salary (all elements work on the same input data).
(d)	A module that logs in the user, then displays the menu, then records the activity — steps done in a fixed order.
(e)	A module that initialises all variables, opens all files, and resets all counters at program start-up.
(f)	A module containing "print error, print report, print salary slip," the action chosen by a flag passed to it.
(g)	A module that groups several completely unrelated functions put together only to fill one file.

C7. Identify the TYPE of coupling (data, stamp, control, external, common, or content) in each case below:

#	**Situation between two modules**
(a)	Module A passes a single integer value to Module B.
(b)	Module A passes a whole record/structure to Module B, which uses only one field of it.
(c)	Module A passes a flag to Module B that controls which action B performs.
(d)	Two modules share and both modify the same global variable.
(e)	Module A branches directly into the middle of Module B / changes B's internal data.
(f)	Two modules depend on the same external file format / communication protocol.

C8. A module currently does three unrelated things — it validates input, sends an email, and writes to the database — and it shares a global status variable with another module. Name the **cohesion** and **coupling** problems present, and explain how you would redesign it to improve both.

PART D — Self-Study : The COCOMO Model (COCOMO 81 & COCOMO II)

Study the primer below, then attempt all questions. Use the formulas and constants exactly as given.

Primer — learn this, then solve D1–D7

1. COCOMO 81 (Barry Boehm, 1981) estimates effort and schedule from software size in KLOC (thousands of delivered lines of code). It has three models of increasing detail — **Basic, Intermediate, Detailed** — and three project modes: **Organic** (small team, familiar problem), **Semi-detached** (medium size, mixed experience), and **Embedded** (large, tightly constrained).

Basic COCOMO equations:

Effort = a × (KLOC)^b person-months (PM)

Time = c × (Effort)^d months People = Effort ÷ Time

Organic: a=2.4 b=1.05 c=2.5 d=0.38 Semi-detached: a=3.0 b=1.12 c=2.5 d=0.35 Embedded: a=3.6 b=1.20 c=2.5 d=0.32

Intermediate COCOMO multiplies the Basic effort by an Effort Adjustment Factor (EAF) — the product of 15 cost-driver ratings (reliability, team skill, tools, etc.).

2. COCOMO II (Boehm et al., 1995/2000) is the modern successor, built for reuse, object-oriented, and iterative/agile development. It has three sub-models matched to how much is known:

- **Application Composition** — earliest stage; sizes prototypes in Object Points.
- **Early Design** — uses Function Points and 7 coarse multipliers to compare architectures.
- **Post-Architecture** — most detailed; uses KSLOC with 17 effort multipliers and 5 scale factors.

Five scale factors (drive the exponent): PREC (precedentedness), FLEX (development flexibility), RESL (risk/architecture resolution), TEAM (team cohesion), PMAT (process maturity).

17 effort multipliers (give the EAF), in four groups: Product, Platform, Personnel, and Project factors.

COCOMO II Post-Architecture equations:

E = 0.91 + 0.01 × ΣSF Effort = 2.94 × (Size)^E × EAF (Size in KSLOC)

F = 0.28 + 0.2 × (E − 0.91) Time = 3.67 × (Effort)^F

People = Effort ÷ Time Productivity = (Size × 1000) ÷ Effort (LOC per PM)

Key difference: in COCOMO 81 the exponent is fixed by the project mode; in COCOMO II it is computed from the scale factors, so economies/diseconomies of scale are modelled. COCOMO II also adds reuse and function-point sizing.

D1. Explain the three models of COCOMO 81 (Basic, Intermediate, Detailed) and its three project modes (Organic, Semi-detached, Embedded).

D2. A project is estimated at **40 KLOC** and is of **Organic** type. Using Basic COCOMO, calculate the Effort, Development Time, and number of persons required. Show all steps.

D3. A project is estimated at **200 KLOC**. Using Basic COCOMO, calculate the Effort and Development Time in **all three modes**, and comment on how the effort changes across the modes.

D4. Explain the three sub-models of COCOMO II (Application Composition, Early Design, Post-Architecture) and state two ways in which COCOMO II improves upon COCOMO 81.

D5. Name the **five scale factors** of COCOMO II and explain the role they play in the effort equation. How do the 17 effort multipliers differ from the scale factors?

D6. A project is estimated at **60 KSLOC**. Using COCOMO II (Post-Architecture) with ΣSF = 14 and EAF = 1.05, calculate: (a) the exponent E, (b) Effort, (c) Development Time, (d) number of persons, and (e) Productivity. Show all steps.

D7. Repeat the COCOMO II calculation for a **120 KSLOC** project with ΣSF = 20 and EAF = 0.95. Then compare its productivity with that of D6 and explain why the two differ.

PART E — Self-Study : Cyclomatic Complexity

Study the primer below, then attempt all questions. For each program, draw the control-flow graph and show your calculation.

Primer — learn this, then solve E1–E7

Cyclomatic complexity (Thomas McCabe) is a software metric that measures the **number of linearly independent paths** through a program by counting its decision points. The higher the value, the more complex and error-prone the code.

Control-Flow Graph (CFG): a directed graph in which each node is a block of statements and each edge is a possible transfer of control; a decision (predicate) node is any if / while / for / case that splits the flow.

Formulas (all give the same value):

1. V(G) = E − N + 2P (E = edges, N = nodes, P = connected components; for one program P = 1, so E − N + 2)
2. V(G) = D + 1 (D = number of decision / predicate nodes)
3. V(G) = number of enclosed regions of the CFG

Worked example: for a single if–else (A=10; if B>C then A=B else A=C; print A,B,C) the CFG has 7 nodes and 7 edges, so V(G) = 7 − 7 + 2 = 2 — two independent paths (the true branch and the false branch).

Meaning: V(G) = number of independent paths = the minimum number of basis-path test cases needed for full path coverage.

Uses: finds the independent paths to test; ensures every path is exercised at least once; shows the minimum testing effort and where to focus; serves as a quality/risk metric to compare designs. Rule of thumb: a module with V(G) above about 10 is too complex and should be split.

E1. Define cyclomatic complexity. State its three formulas and explain what the resulting number represents for software testing.

E2. For the program below: (a) draw the control-flow graph, (b) compute V(G) by all three methods, (c) list the independent paths, and (d) state how many test cases are required.

```
1.  sum = 0
2.  i = 1
3.  while (i <= n)
4.      sum = sum + i
5.      i = i + 1
6.  end while
7.  if (sum > 100)
8.      print "Big"
    else
9.      print "Small"
10. end
```

E3. Repeat parts (a)–(d) for the program below.

```
1.  read marks
2.  if (marks >= 90)
3.      grade = 'A'
4.  else if (marks >= 60)
5.      grade = 'B'
    else
6.      grade = 'C'
7.  print grade
```

E4. Repeat parts (a)–(d) for the nested-decision program below (find the maximum of three numbers).

```
1.  read a, b, c
2.  if (a > b)
3.      if (a > c)
4.          max = a
        else
5.          max = c
6.  else
7.      if (b > c)
8.          max = b
        else
9.  max = c
10. print max
```

E5. State any four uses / significance of cyclomatic complexity in software testing and quality assurance.

E6. State the threshold value commonly used as an upper limit for cyclomatic complexity, and explain the relation between V(G) and the number of test cases required.

E7. A module is measured and found to have a cyclomatic complexity of 18. What does this tell you about the module, and what action would you recommend?

Important Note

Although Parts D and E are given here as self-study topics, both **COCOMO II** and **Cyclomatic Complexity** are part of the examination syllabus and must be studied in detail for the exam — including their theory, formulas, and numerical problems.

——— END OF ASSIGNMENT ———